# Secure Login System

(PHP | HTML | CSS | SQL | Secure Authentication | SQL Injection)

By George Nicholl

The Secure Login System is a lightweight, PHP-based authentication module designed to demonstrate strong capability in both software engineering and cybersecurity. It provides core functionality for user registration, login, and session-based access to protected content, while emphasising secure coding practices and a defence-in-depth approach to web security.

The system is intentionally framework-free, using native PHP and PDO to give full control over how data is handled, how logic is structured and how security controls are applied. This makes it suitable both as a teaching tool and as a reusable authentication component in small to medium projects.

## Project Objectives

This project was built with dual objectives: to be a clean, maintainable piece of backend software, and to follow security practices aligned with modern industry standards.

From a software engineering perspective, the objectives were:

- To design a modular system with a clear separation between presentation and logic.
- To structure the project so that it can be easily extended (e.g. adding roles, profiles, or 2FA).
- To keep the codebase understandable, readable and easy to maintain

From a cybersecurity perspective, the objectives were:

- To avoid common authentication vulnerabilities such as SQL injection, weak password storage, and insecure session handling.
- To minimise the attack surface through careful directory structure and access restrictions.
- To ensure that user data, particularly credentials, is handled securely at every stage of the authentication process.

## Technology Stack

| Component | Description |
|---|---|
| Languages Used | PHP, HTML5, .htaccess, enforcement |
| Web Server | Apache (with .htaccess enforcement) |
| Database | MySQL / MariaDB |
| Security Methods | Prepared Statements (PDO), password hashing (Bcrypt), sanitisation |
| Architecture | Modular PHP with public/private separation |
| Tools / IDE | Visual Studio Code, XAMPP / WAMP / LAMP |
| Development Approach | Secure-by-design, modular, maintainable code structure |

This technology stack was intentionally minimal – using no external PHP frameworks to strengthen backend fundamentals and ensures the security model is fully visible and under direct control.

## High-Level Architecture

The secure Login System uses a two-tier public/private architecture. All user-facing pages live in a /public directory, while the sensitive backend logic and configuration are placed inside a /private directory that is not directly accessible via the web server.

When a user visits the site, they only ever interact with scripts inside the /public. These scripts then internally include and call functions from /private where databases access, input validation and authentication logic are handled. This architecture enforces a clear boundary between the presentation layer and sensitive logic, reducing the system's exposed attack surface.

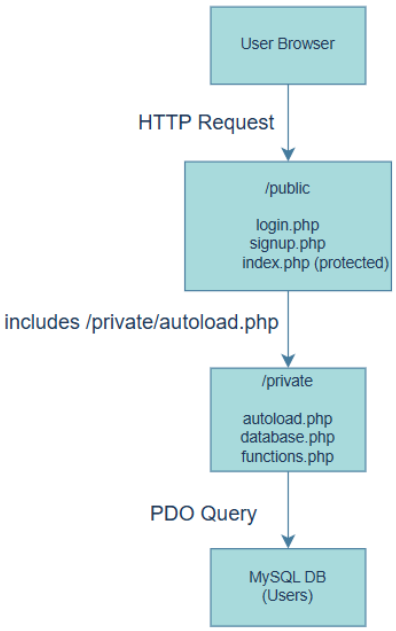


*Figure 1: High-Level System Architecture*

**Directory Structure & Code Organisation**

This project's folder structure is organised to support both clarity and security:

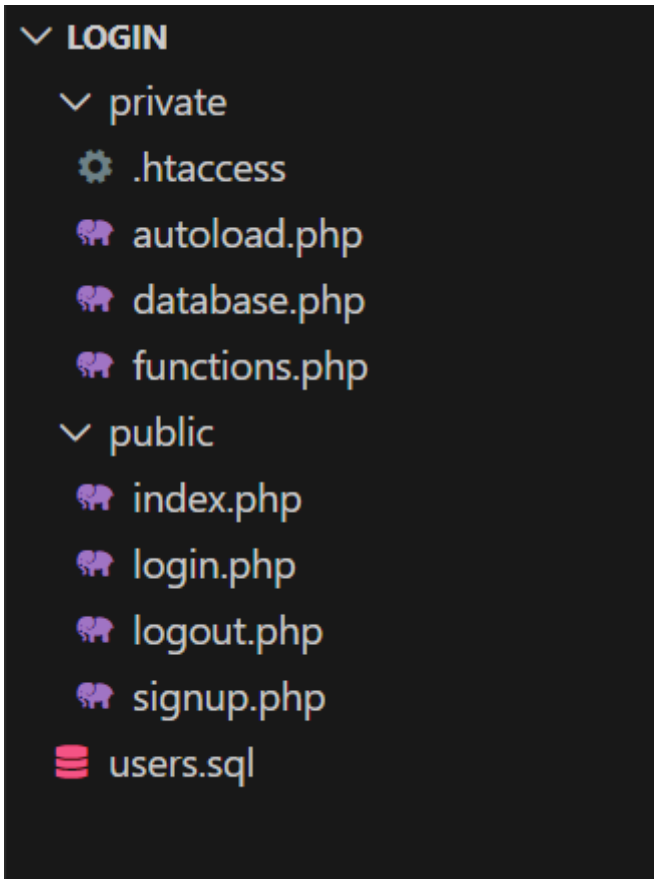


*Figure 2: Project Folder Structure*

The /public directory contains all pages that can be requested directly by the user. These files handle form display and basic routing.

The /private directory contains the core logic:

- Database.php – Creates the PDO database connection.
- Functions.php – Contains all reusable utility and authentication-related functions.
- Autoload.php – Central file that starts the session and loads common dependencies
- .htaccess – Prevents direct web access to the entire /private directory.

This structure ensures that even if an attacker discovers the file paths, they cannot directly execute or download sensitive backend scrips.

```
private > autoload.php
1   <?php
2   session_start();
3
4   require "../private/database.php";
5   require "../private/functions.php";
```

*Figure 3: Secure Autoload Pattern*

The screenshot from figure 3 demonstrates the centralised loading of the database and helper functions with an early session start.

## Database Design

The database is intentionally minimal and designed around secure credential handling. The system uses a single table, users, to store account information.

Typical Structure:

- Id – Primary key (INT, auto-increment)
- Username – Chosen Display name
- Email – Login identifier (unique)
- Password – Hashed password (Bcrypt)
- url_address – A random identifier (e.g, used as a profile URL key)
- created_at – Timestamp indicating when the user registered.

This design follows data minimisation: only the fields required for authentication and basic identification are stored. Passwords are never stored in plaintext; they are hashed using a one-way algorithm, so that even if the database is compromised, the actual passwords are significantly harder to recover.

| Users |
| --- |
| id (INT, Primary key) \| username (VARCHAR) \| email (VARCHAR, Unique) \| password (VARCHAR, Hashed) \| url_address (VARCHAR) \| created_at (TIMESTAMP) |

*Figure 4: Users Table*

## Authentication Workflow

The login workflow follows a multi-step, security-aware sequence:

1. The user submits their email and password via login.php
2. The email and password are validated and sanitised server-side
3. A parameterised SQL query is executed using PDO to safely look up the user record by email (and optionally url_address).
4. If a record exists, the system uses password_verify() to compare the submitted password with the stored hash.
5. On success, the system regenerates the session ID and stores minimal user information in the session (such as user_id and username)
6. The user is redirected to protected page (index.php) that checks for an active session before rendering.
7. On failure, a generic error message is returned, avoiding disclosure of which part of the credentials was incorrect.

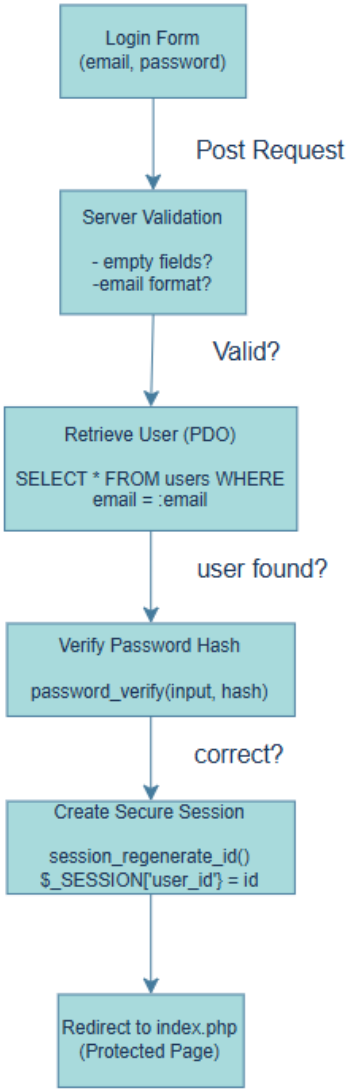


*Figure 5: Authentication Workflow Diagram*

```
74  </style>
75  <form method="post">
76      <div><?php
77          if(isset($Error) && $Error != "");
78          {
79              echo $Error;
80          }
81       ?></div>
82      <div id="title">Login</div>
83      <input id="textbox" type="email" name="email" required><br>
84      <input id="textbox" type="password" name="password" required><br><br>
85      <input type="submit" value="Login">
86  </form>
87  </body>
88  </html>
```

*Figure 6: Login Form Structure - Code Screenshot*

The screenshot in figure 6 demonstrates a minimal, controlled input surface for authentication.

## User Interface & Front-End Layout

While the Secure Login System is primarily a backend-driven project, the user interface plays an important role in ensuring that the authentication process is clear, intuitive and accessible. The system adopts a minimal and functional design approach, prioritising usability and clarity while avoiding unnecessary styling complexity. This ensures the interface aligns with the system's purpose: secure, reliable user authentication with a clean and distraction-free experience.

The front end is written using HTML and CSS without using external UI frameworks. This decision keeps the implementation lightweight and transparent, allowing full control over both behaviour and styling. Each page maintains a simple centred layout that positions the login and signup forms at the top of the user's visual focus, ensuring that the user journey is straightforward and predictable.

The login page (login.php) contains two main input fields – email and password – both of which use HTML5 validation attributes such as required and type="email" to provide immediate client-side guidance. This reduces the frequency of malformed submissions before server-side validation takes place. Error messages produced during failed login attempts (for example, incorrect credentials or invalid input) are displayed neatly above the form, providing timely feedback while avoiding exposure of sensitive system behaviour or authentication logic.

Similarly, the signup page (signup.php) mirrors the visual structure, creating a consistent user experience throughout the authentication journey. Both pages are

styled using a shared CSS file (style.css), ensuring consistency in typography, spacing and layout.

After successful authentication, the user is redirected to the protected page (index.php), which displays personalised content based on the authenticated session. This page servers as confirmation that the login process succeeded and that session-based access control Is functioning correctly.

The following figures illustrate the appearance of the system's front-end pages:

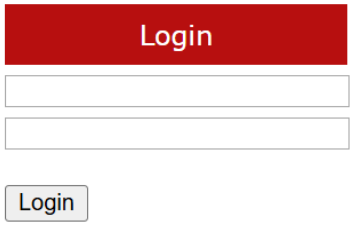


*Figure 7: Login Page Interface*

*Figure 8: Login Page Showing Error Message*

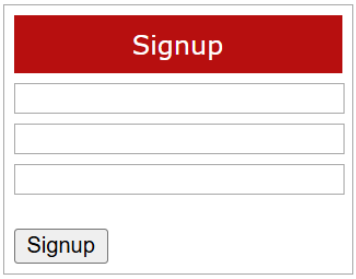


*Figure 9:Signup Page Interface*

Hi george
This is the Home page

Logout

*Figure 10: Protected Page after Successful Login*

## Secure Coding Principles

The Secure Login System implements several industry-standard security practices to ensure that user authentication is handled safely and that common attack vectors such as SQL injection, credential theft, and session hijacking are effectively mitigated. This section outlines the core secure coding principles used within the system and provides illustrative code snippets demonstrating their implementation.

These principles collectively ensure that the authentication is handled safely, user credentials remain protected and the backend cannot be manipulated through malicious input.

### Server-Side Input Validation

Although HTML provides basic client-side validation, attackers can bypass browser checks. For this reason, the system implements server-side validation in login.php to ensure that the only properly structured email addresses and non-empty passwords are processed.

The validation logic checks for empty values and confirms proper email formatting using PHP's filter_var() function.

```php
if($_SERVER['REQUEST_METHOD'] == "POST")
{
    $email = $_POST['email'];
    if(!preg_match("/^[\w\-]+@[\w\-]+.[\w\-]+$/", $email))
    {
        $Error = "Please enter a valid email";
    }

    $password = $_POST['password'];
```

*Figure 11:Server-Side validation of email and password in login.php*

Protection Against SQL Injection (Prepared Statements)

To defend against SQL injection, the system uses prepared statements through PHP's PDO interface. Instead of directly concatenating user-supplied values into SQL strings, placeholders are used and bound securely to variables. This ensures that user input treated strictly as, not executable SQL.

The login process retrieves a user record based on the submitted email using a prepared SELECT query. The password is never included in the WHERE clause as plaintext.

```php
    if(isset($_SESSION['url_address'])) {
        $arr['url_address'] = $_SESSION['url_address'];
        $arr['email'] = $email;

        $query = "SELECT * FROM users WHERE url_address = :url_address AND email = :email LIMIT 1";
        $stm = $connection->prepare($query);
        $check = $stm->execute($arr);

        if($check){
            $data = $stm->fetchAll(PDO::FETCH_OBJ);
            if(is_array($data) && count($data) > 0){
                return $data[0];
            }
        }
    }
```

*Figure 12: Prepared SQL Statement used for secure user lookup*

Secure Password Hashing

During user registration, the system uses PHP's native password_() function to securely transform user passwords into one-way, salted hashes before storing them in the database. This ensures that even if the users table were compromised, the original passwords cannot be easily recovered.

Below, figure 13 illustrates the password hashing logic implemented in signup.php. The figure highlights the point at which the raw password submitted via the registration form is processed server side, transformed into a secure hash, and then inserted into the database using a prepared statement

```php
$username = esc($username);
$password = esc($_POST['password']);
```

*Figure 13: Password hashing during user registration (signup.php)*

Figure 13 shows the password being retrieved from the POST request, securely hashed using PHP's password_hash() function, and passed as a bound parameter in the SQL INSERT statement. This ensures that plaintext passwords are never stored in the database.

By using a one-way hashing algorithm with automatic salting, the system significantly reduces the impact of potential database breaches. Even with full access to the stored hashes, an attacker would be unable to feasibly reverse them to obtain the original passwords.

This method aligns with modern authentication best practices and prevents common security failures associated with plaintext or weakly encrypted password storage.

Session Management and Access Control

In addition to validating input and protecting database access, the Secure Login System relies on PHP sessions to maintain authenticated state between page requests. Sessions are used to securely remember which user is logged in, without ever exposing sensitive data such as passwords or raw database identifiers in the URL.

A central autoload.php file is included at the top of all public-facing pages to ensure that session_start() is called in a single, controlled place. This avoids duplicated session logic and ensures that every request is consistently tied to the correct session.

Once a user has successfully authenticated, a small amount of information (e.g. the users ID and username) is stored in the $_SESSION superglobal. The protected page (index.php) checks whether this session data is present before rendering any content. If no valid session is found, the request is redirected back to login.php, preventing unauthorised access.

This session-based access control ensures that:

- Only authenticated users can load protected content.
- Direct URL guessing e.g. navigating straight to index.php does not bypass authentication.
- The application does not rely on insecure mechanisms such as hidden form fields or query parameters to track login state.

```
public > index.php
<?php

    require "../private/autoload.php";

    require_login();
?>

<!DOCTYPE html>
<html>
<head>
    <title>Home</title>
</head>
<body>
    <div id="header">
        <div>Hi <?= esc_html($_SESSION['username'] ?? '') ?>!</div>
        <div style="float:right">
            <a href="logout.php">Logout</a>
        </div>
    </div>

    This is the Home page
</body>
</html>
```

*Figure 14: screenshot of session-based access control check in index.php*

The figure above shows the conditional logic that verifies the presence of an active session before loading the protected content and redirects unauthenticated requests back to the login page.

<u>Directory and File Access Restrictions (.htaccess)</u>

A critical principle applied in this project is that sensitive logic should never be directly accessible from the web. To enforce this, the project is split into a /public and /private directory structure.

- /public contains all files that can be requested directly by the browser (e.g.) login.php, signup.php, index.php)
- /private holds configuration files, database connection logic, shared functions and other backend-only code.

An .htaccess file located inside /private is used to block direct HTTP access to that directory. Typical rules include a blanket deny directive so that if an attacker discovers the path to a private file, it still cannot be downloaded or executed via the browser.

This arrangement provides and additional layer of defence beyond input validation:

- Database credentials and internal helper functions cannot be exposed by misconfigured routing.
- Even if an attacker attempts to access /private/database.php or /private/functions.php directly, the web server will refuse the request.
- All interaction with the private code must go through carefully controlled entry points in /public.

```
private > .htaccess > ...
Options -Indexes

<IfModule mod_authz_core.c>
    Require all denied
</IfModule>

<IfModule !mod_authz_core.c>
    Deny from all
</IfModule>
```

**Forbidden**

You don't have permission to access this resource.

*Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12 Server at localhost Port 80*

*Figure 15: htaccess rule preventing web access to the /private directroy*

Error Handling and Generic Feedback

Error messages are a subtle but important part of secure coding, overly detailed errors can leak information about the internal structure of the system, while vague or missing feedback can frustrate legitimate users.

The Secure Login System follows a “generic but useful” error-handling approach:

- When login fails, the system returns a single, non-specific message such as “Wrong email or password” rather than revealing which field was incorrect.
- Validation errors (e.g. missing email or invalid format) are displayed clearly above the form but do not include any stack traces, SQL fragments or internal file paths.

- Database or unexpected server errors are not sent directly to the browser. Instead, the user sees a controlled message while the root cause can be investigated separately in the development environment.

By avoiding overly specific authentication errors (e.g. “email not found” or “password is incorrect”) the system reduces the risk of:

- Username enumeration attacks, where an attacker tries different emails and observes which ones “exist”.
- Information Leakage, where internal details of the database or file system might accidentally be exposed.

```
{
    $Error = "Please enter a valid email";
}
```

*Figure 16: Example of generic login error message displayed to user*

Figure 16 above shows how validation and authentication errors are rendered above the form in a user friendly but security-conscious way.

<u>Output Encoding and XSS Mitigation</u>

While this project focuses primarily on secure authentication and database access, basic protections against Cross-Site Scripting (XSS) are also considered. Whenever user-supplied data is displayed back in the browser (e.g. a username shown on the protected page), it should be safely encoded before output.

A common pattern is to route all output intended for HTML through a small helper function (often called something like esc() ), which wraps PHP’s htmlsepcialchars() function. This converts characters such as <, >, and into their HTML-encoded equivalents so that they cannot be interpreted as executable script by the browser.

This approach ensures that:

- If an attacker attempts to inject JavaScript via a form field (e.g. in the username), it will be rendered as plaintext.
- The risk of stored or reflected XSS in simple text outputs is significantly reduced.
- The codebase has consistent, central place where output encoding can be maintained or extended in the future.

```php
// Eascapes ouput to prevent XSS when rendering user-controlled data
function esc_html(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITURE, 'UTF-8');
}
```

```php
    <div id="header">
        <div>Hi <?= esc_html($_SESSION['username'] ?? '') ?>!</div>
        <div style="float:right">
            <a href="logout.php">Logout</a>
        </div>
```

*Figure 17: Use of an output-escaping helper function before rendering user data from functions.php & index.php*

Figure 17 shows a helper function that wraps htmlspecialchars() and is applied to values before they are echoed into the HTML response.

Defence-in-Depth Summary

Taken together, these secure coding practice form a defence-in-depth model:

- Validation stops malformed or clearly malicious input early.
- Prepared statements protect database queries from injection.
- Directory separation and .htaccess reduce the exposed attack surface.
- Sessions and access checks ensure only authenticated users reach protected content.
- Safe error messages and encoded output limit information leakage and XSS risk

These controls work together so that even if one layer is bypassed or misconfigured, other layers still provide protection and make successful exploitation significantly more difficult.

Together, these measures ensure that security is not dependent on any single mechanism, aligning with the project's goals of building a resilient and maintainable authentication system.

## Data Flow through the System

Understanding how data moves through the Secure Login System is essential to demonstrating both its functional correctness and its security properties. This section outlines the end-to-end flow of data from the moment a user submits credentials to the point where authenticated access is granted or denied.

The data flow is intentionally linear and controlled, ensuring that user input is validated, processed, and stored securely at each stage.

Step 1: Client-Side Input

The process begins when a user enters their email and password into either the login or signup form. These forms are saved from the /public directory and use basic HTML5 validation attributes such as ‘required’ and ‘type=”email”’ to provide immediate feedback and reduce malformed submissions.

Although client-side validation improves usability, it is not relied upon for security.

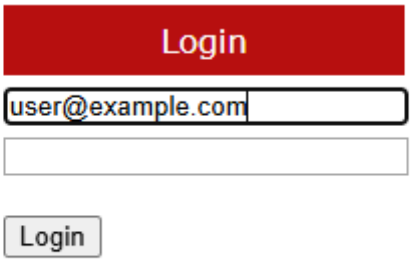


*Figure 18: Login request initiated from the browser*

Step 2: Public PHP Entry Point

When the form is submitted, the request is sent via HTTP POST to login.php or signup.php in the /public directory. These files act as controlled entry points into the system.

At this stage:

- Input values are read from $_POST
- Server-side validation is applied
- Errors are handled before any database interaction occurs

```php
<?php

require "../private/autoload.php";
$Error = "";

if($_SERVER['REQUEST_METHOD'] == "POST")
{
    $email = $_POST['email'];
    if(!preg_match("/^[\w\-]+@[\w\-]+.[\w\-]+$/", $email))
    {
        $Error = "Please enter a valid email";
    }
```

*Figure 19: Public entry point receiving POST data*

Step 3: Autoload and Backend Initialisation

Each public script includes a central autoload.php file from the /private directory. This file is responsible for:

- Starting the session
- Loading shared helper functions
- Initialising the database connection

This ensures consistent behaviour across all pages and avoids duplicated setup logic.

```
private > autoload.php
1   <?php
2   session_start();
3
4   require "../private/database.php";
5   require "../private/functions.php";
```

*Figure 20: autoload.php initialising sessions and dependencies*

Step 4: Database Interaction

Once input has been validated, the system interacts with the database using prepared statements via PDO. Queries are parameterised, and user input is never concatenated directly into SQL strings.

Depending on the action:

- Login attempts retrieve a user record by email
- Signup attempts insert a new user record

```
if($Error == ""){

    $arr['password'] = $password;
    $arr['email'] = $email;

    $query = "SELECT * FROM users WHERE email = :email AND password = :password LIMIT 1";
    $stm = $connection->prepare($query);
    $check = $stm->execute($arr);
```

*Figure 21: Secure PDO query execution*

Step 5: Authentication Decision

If a matching user record is found:

- Credentials are verified
- A session is established
- The user is redirected to a protected page

If authentication fails:

- A generic error message is returned
- No internal state or validation details are exposed.

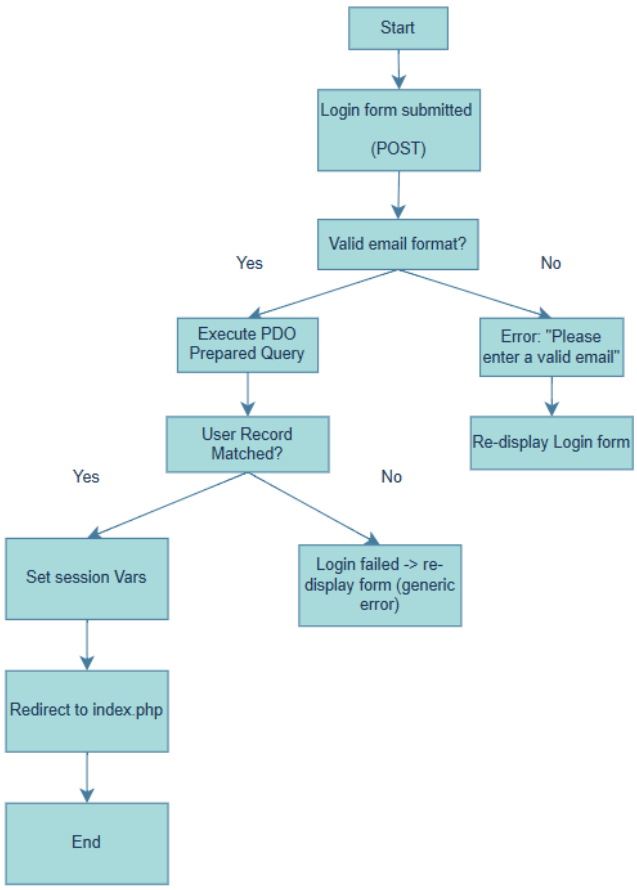


*Figure 22: Authentication Success VS Failure Logic*

Step 6: Session-Based Access Control

Once authenticated, subsequent requests rely on session data rather than repeated credential submission. Protected pages check for an active session before rendering content.

If no valid session exists, access is denied and the user is redirected to the login page.

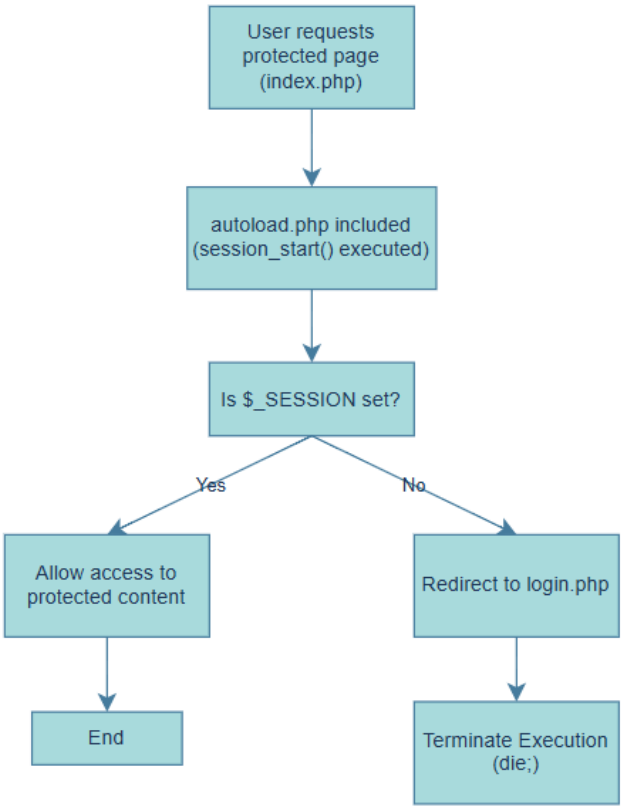


*Figure 23: Session-based access control flow*

## Threat Mitigation and Security Controls

The Secure Login System has been designed to mitigate common web application threats through a layered security approach. Rather than relying on a single control, multiple defences are applied throughout the system.

SQL Injection

Mitigated through:

- PDO prepared statements
- Parameter Binding
- No dynamic SQL construction

Unauthorised Access

Mitigated through:

- Session-based access control

- Authentication checks on protected pages
- Redirects for unauthenticated requests

Information Disclosure

Mitigated through:

- Generic error messages
- No exposure of SQL queries or file paths
- Private Directory access restrictions

Cross-Site Scripting (XSS)

Mitigated through:

- Output encoding using helper functions
- No direct rendering of raw user input

| **Threat** | **Description** | **Implemented Controls** | **Relevant Section** |
|---|---|---|---|
| SQL Injection | Malicious Input Designed to manipulate database queries | PDO prepared statements with bound parameters; no dynamic concatenation | Secure Coding principles – Prepared Statements |
| Cross Site Scripting (XSS) | Injection of malicious scripts via user-supplied input | Output encoding using htmlspecialchars() via helper function | Secure Coding Priniples – Output Encoding |
| Session Hijacking | Attacker attempts to reuse or steal an authenticated session. | Server-side session storage; session-based access checks on protected pages | Secure Coding Principles – Session Management |
| Unauthorised Direct Access | Accessing protected pages via direct URL navigation | Session existence checks and forced redirects | Session-Based Access Control |
| Information Disclosure | Leakage of system or authentication details. | Generic error messages; no SQL or stack traces exposed | Error handling and Feedback |
| Directory Traversal / File Access | Direct access to backend files via URL guessing | Private directory protected by .htaccess | Directory Structure & Access Restrictions |

*Figure 24: Threats mapped to implemented controls*

## Testing & Validation

Testing was conducted throughout development to ensure that the system behaves correctly under both normal and malicious conditions.

Functional Testing

The following behaviours were tested manually:

| Test Case | Expected Result |
|---|---|
| Valid Login | Redirect to protected page |
| Invalid Login | Generic error message |
| Direct access to index.php | Redirect to login |
| Duplicates signup | Error message |
| Logout | Session Destroyed |

Manual Functional Testing Outcomes

| Test ID | Test Scenario | Expected Outcome | Actual Outcome | Result |
|---|---|---|---|---|
| FT-01 | Submit login form with valid email and password | User is authenticated and redirected to protected page | User successfully redirected to index.php | Pass |
| FT-02 | Submit login form with invalid email format | Validation error displayed | Error message displayed above login form. | Pass |
| FT-03 | Submit login form with incorrect credentials | Login denied with generic error | Login denied, user remains on login page | Pass |
| FT-04 | Access index.php without an active session | User redirected to login page | Redirect to login page occurs | Pass |
| FT-05 | Register a new user account | User details stored and account created | New user successfully added to database | Pass |
| FT-06 | Attempt to register using an existing email | Registration rejected with error | Error message displayed | Pass |
| FT-07 | Logout from authenticated session | Session destroyed and access revoked | User Redirected to login page | Pass |
| FT-08 | Refresh protected page after logout | Access Denied | Redirect to login page | Pass |

*Figure 25: Manual functional testing outcomes*

Security Testing

Security-focussed tests included:

- SQL injection attempts in input fields
- Direct URL access to private files
- Invalid session manipulation

All attempts were safely blocked by existing controls.

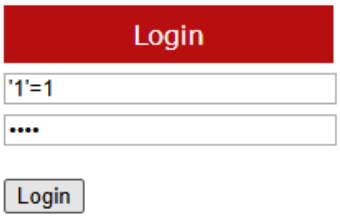


*Figure 26(a): SQL Injection payload entered into the login form*

```
$arr['password'] = $password;
$arr['email'] = $email;

$query = "SELECT * FROM users WHERE email = :email AND password = :password LIMIT 1";
$stm = $connection->prepare($query);
$check = $stm->execute($arr);
```

*Figure 26(b): A screenshot of the prepared statement block above*

## Limitations and Future Improvements

While the Secure Login System demonstrates strong foundational security practices, several limitations remain that would need to be addressed before deployment in a production environment.

Current Limitations

- No password reset functionality
- No multi-factor authentication
- No rate limiting on login attempts
- No CSRF tokens on forms
- Basic user interface styling only

Future Improvements

Potential enhancements include:

- Implementing two-factor authentication (2FA)
- Adding CSRF protection tokens
- Introducing account lockout or throttling
- Email verification on registration
- Role-based access control

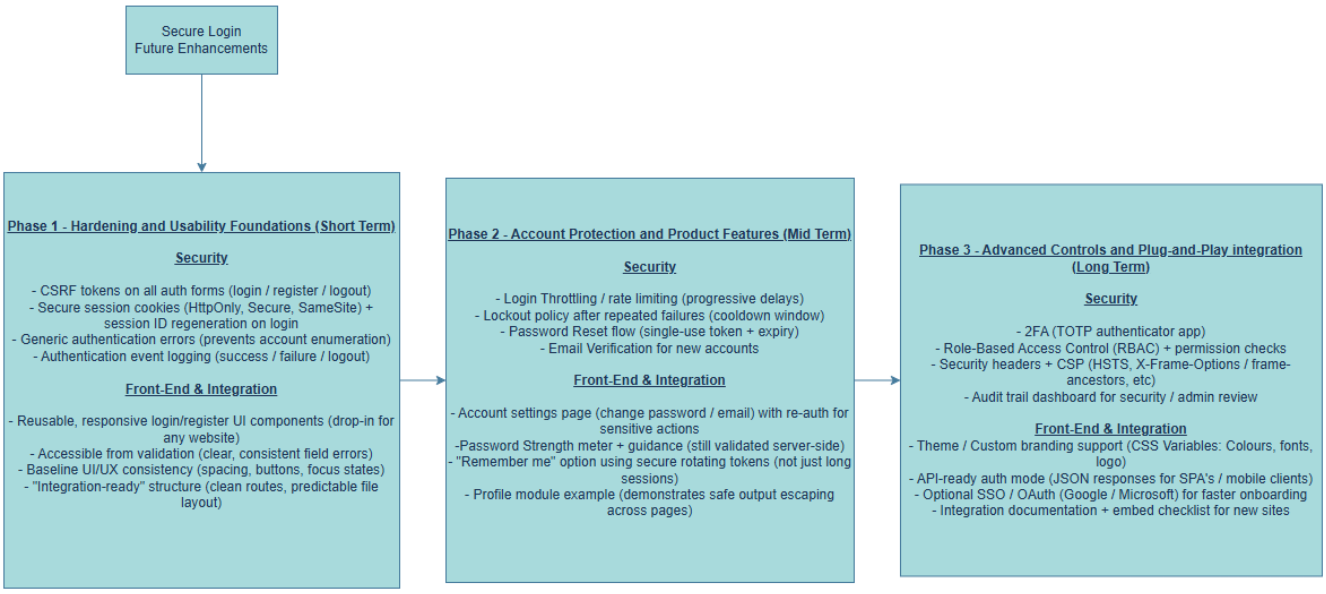


*Figure 27: Proposed future security enhancements roadmap*

## Conclusion

The Secure Login System demonstrates a practical implementation of secure authentication principles using native PHP and industry-recognised best practices. Through careful architecture design, controlled data flow, and layered security controls, the system successfully mitigates common web applications such as SQL injection, unauthorised access, and information leakage.

Beyond its immediate functionality, this project serves as a strong learning artefact, highlighting an understanding of secure coding, backend architecture, and defensive programming. Its modular structure allows for future expansion, making it suitable as both a teaching tool and a foundation for more advanced authentication systems.